



El Arte del Control del Sistema

Gestión de Procesos en Linux: De la observación pasiva a la administración total



Visibilidad



Flujo



Terminación



Optimización

Este material transforma al usuario en administrador. Aprenda a monitorear la 'caja negra', manipular el tiempo, decidir la vida de los procesos y optimizar.

La Instantánea Estática: Estática: Comando `ps`

El comando **`ps`** (Process Status) captura el estado de los procesos en el momento exacto de la ejecución. Es la herramienta fundamental de diagnóstico, funcionando como una fotografía estática del sistema.

Diferencia clave: **`ps`** es una foto; **`top`** es un video.

```
$ ps aux | grep jose
```

USER	PID	%CPU	%MEM	COMMAND
jose	1234	0.0	0.1	/usr/bin/gnome-shell

```
$
```

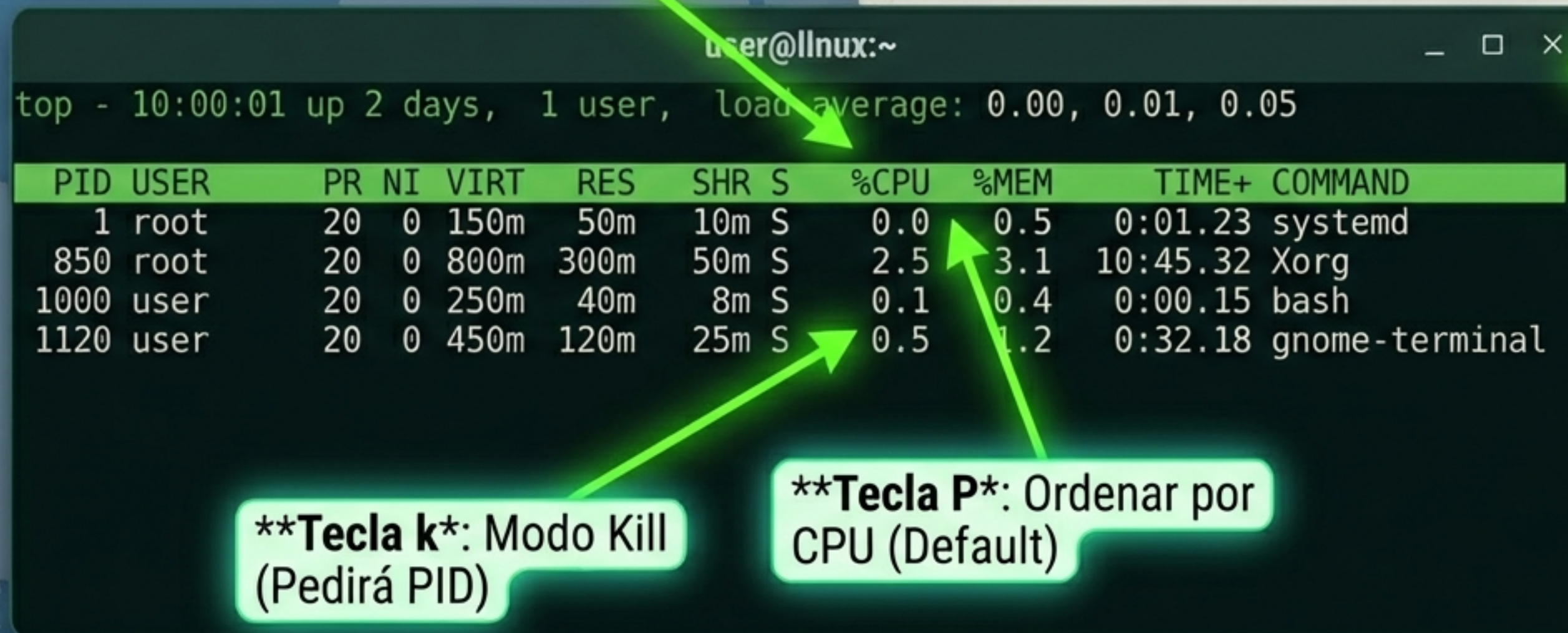
ps aux: Muestra todos los procesos (a), incluso sin terminal (x), con usuario (u).
ps ef: Muestra la jerarquía de árbol.

El Monitor en Tiempo Real: Comando `top`

Visualización dinámica e interactiva del sistema.

****Tecla M**:** Ordenar por Memoria

****Tecla q**:**
Salir



```
user@linux:~  
top - 10:00:01 up 2 days, 1 user, load average: 0.00, 0.01, 0.05
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	150m	50m	10m	S	0.0	0.5	0:01.23	systemd
850	root	20	0	800m	300m	50m	S	2.5	3.1	10:45.32	Xorg
1000	user	20	0	250m	40m	8m	S	0.1	0.4	0:00.15	bash
1120	user	20	0	450m	120m	25m	S	0.5	1.2	0:32.18	gnome-terminal

****Tecla k**:** Modo Kill
(Pedirá PID)

****Tecla P**:** Ordenar por
CPU (Default)

La Evolución Visual: Comando `htop`

Una alternativa moderna a `top`. Ofrece desplazamiento vertical/horizontal, uso del ratón y gráficos de barras intuitivos para CPU y Memoria.



Navegación con flechas y ratón

F6: Menú de ordenamiento y filtros

F9: Acceso directo a `kill`

Requiere instalación: `sudo apt install htop`

Gestión del Flujo: Primer y Segundo Plano

Primer Plano (Foreground)

```
user@linux:~$ Proceso en Ejecución (sleep 3600)
```

El usuario debe esperar hasta que el proceso termine. La terminal está bloqueada.

Segundo Plano (Background)

```
user@linux:~$ sleep 3600 &  
[1] 12345  
Proceso en Ejecución (sleep 3600)
```

Segundo Plano (Background)

Ejemplo de Código y Salida

Input: `sleep 3600 &`

Output: `[1] 12345`

[1] = Job ID (Identificador de trabajo)

12345 = PID (Identificador de proceso del sistema)

Congelando el Tiempo: `Ctrl+Z` y `jobs`

Diferencia vital: **Ctrl+C** cancela y mata. **Ctrl+Z** envía la señal SIGTSTP y suspende el proceso.

```
user@linux:~$ nano prueba.txt
^Z
[2]+  Stopped                  nano prueba.txt
user@linux:~$ jobs
[1]-  Running                  sleep 3600 &
[2]+  Stopped                  nano prueba.txt
user@linux:~$
```

Señal **SIGTSTP** envía al proceso a segundo plano (suspendido)

Insight:
El signo **+** indica el trabajo actual por defecto.

Recuperando el Control: ``fg`` y ``bg``

Command Definitions

- ``fg %N``: Trae el trabajo al Primer Plano (Foreground).
- ``bg %N``: Reanuda el trabajo en Segundo Plano (Background).

Step 1:
User types ``bg %2``
for ``nano``.

Step 2: ``nano`` tries
to write to TTY
(Terminal).

**Step 3: Kernel
Intercepts!**

Signals **SIGTTIN/SIGTTOU**.

Los procesos interactivos
(editores) siempre deben
manejarse con ``fg``.

Step 4:
Process stops
immediately.

Anatomía de una Terminación: `kill`

No es solo matar, es enviar señales. Elija con sabiduría.



SIGTERM (-15)

La Petición Cortés (Default)

El proceso recibe la señal, guarda datos, cierra archivos y termina limpiamente.

```
kill 12345
```

****Siempre intentar primero.****



SIGKILL (-9)

El Apagón Forzado

Brutal. El kernel elimina el proceso inmediatamente. Sin limpieza. Riesgo de corrupción.

```
kill -9 12346
```

****Último recurso.****

Ejecución Masiva: Comando `killall`

Cerrar todas las instancias de un programa (ej. gedit) sin buscar PIDs individuales.



¡Advertencia de Seguridad! `killall -9 firefox` forzaría el cierre inmediato de todas las ventanas del navegador. ¡Riesgo de pérdida de datos! 🔥



La Jerarquía de Recursos: `nice`

La prioridad de CPU se gestiona con el valor 'Niceness'. Curiosamente, un número más alto significa 'más amable' (menos prioridad).

Escala de Niceness



```
`nice -n 15 tar -czf backup.tar.gz` (Backup silencioso)
```

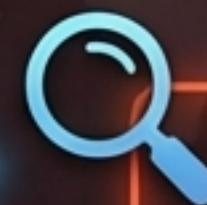

Corrección en Vuelo: Comando `renice`

Modifica la prioridad de un proceso que *ya* se está ejecutando.

 **Problema:** Script Python consumiendo 100% CPU.

Paso 1: Identificar PID

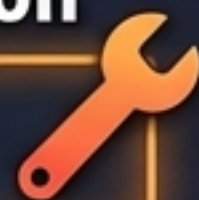
```
ps aux | grep python
```



PID: 12347

Paso 2: Acción de Corrección

```
renice 10 -p 12347
```

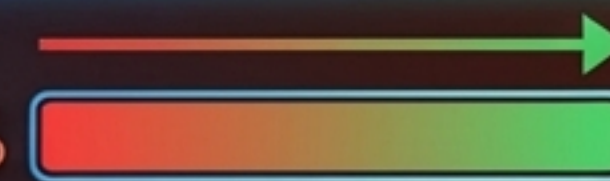


Resultado

```
0. ->| users - _BoT_
```

Prioridad cambia de 0 a 10.
El sistema respira.

100%



low



La Cadena de Montaje: Comando `xargs`

Convierte la salida estándar (STDOUT) en argumentos para el siguiente comando.



Pro Tip: Usar `-p` para pedir confirmación antes de borrar.

Referencia Rápida de **Comandos**

Categoría	Comandos Clave	Acción
Monitor	<code>ps aux</code> , <code>top</code> , <code>htop</code>	Foto estática vs. Monitor en vivo.
Job Control	<code>jobs</code> , <code>Ctrl+Z</code> , <code>bg</code> , <code>fg</code>	Pausar, listar y mover entre planos.
Señales	<code>kill [PID]</code> , <code>killall</code>	Enviar SIGTERM (-15) o SIGKILL (-9).
Prioridad	<code>nice</code> , <code>renice</code>	Ajustar valor Niceness (-20 a 19).
Auto	<code>xargs</code>	Convertir salida en argumentos.

La Responsabilidad del **Administrador**



- **Visibilidad antes de Acción:** Siempre verificar con ``ps`` o ``top``.
- **Cortesía antes de Fuerza:** Usar ``SIGTERM`` antes de ``SIGKILL``.
- **Optimización antes de Terminación:** Usar ``renice`` si el proceso es solo lento.

Dominar el control de procesos es la diferencia entre reiniciar el servidor y solucionar el problema.